Pix regulamentação

Pesquisas (Literatura branca/cinzenta)

https://tacnode.io/post/real-time-fraud-detection-architecture#:~:text=Real%2DTime%20Fraud%20Detection%20Architecture%3A%20Where%20Coherence%20Breaks,fast%20in%20isolation.%20The%20diagram%20looks%20right.

Bolton & Hand (2002), Statistical Fraud Detection: A Review, Statistical Science, vol. 17
Um dos pontos conceituais mais importantes do artigo é a distinção de quatro níveis operacionais de detecção de anomalias:

1. **Individual / Específica (Comportamento histórico do próprio indivíduo):**
    - Modela-se o padrão habitual de uma conta específica (ex.: valores típicos gastos, locais frequentes). Uma transação que destoa muito do perfil histórico pessoal do usuário dispara um alerta.
2. **Comparativa / Por Grupos de Pares (*Peer Group Analysis*):**
    - Compara-se o indivíduo a um grupo de pares semelhantes (ex.: clientes com mesma faixa de renda ou mesmo segmento de negócio). Se o comportamento de alguém começa a divergir drasticamente de seus pares, gera-se suspeita.
3. **Ponto de Ruptura / Mudança Estrutural (*Break Point Analysis*):**
    - Foca na análise temporal de séries temporais ou fluxos de transações para identificar o momento exato em que o padrão de uma conta mudou abruptamente (muitas vezes indicando roubo de cartão ou invasão de conta).
4. **Baseada em Rótulos / Transações Externas:**
    - Comparação com listas negras de entidades suspeitas (números de telefone, endereços de entrega fraudulentos, etc.).

A comprehensive review on financial fraud detection using machine learning and deep learning techniques
Os algoritmos baseados em *Gradient Boosting* (especialmente LightGBM e XGBoost) continuam sendo o padrão da indústria para dados transacionais tabulares devido à sua alta velocidade de inferência, facilidade de ajuste e excelente desempenho em dados estruturados.

Dal Pozzolo, Caelen, Johnson & Bontempi (2015), Calibrating Probability with Undersampling for Unbalanced Classification, IEEE CIDM.
Ver desbalanceamento de classes abaixo

Dal Pozzolo et al. (2018), Credit card fraud detection: a realistic modeling and a novel learning strategy, IEEE TNNLS, 29(8). Trata rótulos atrasados: a fraude só é

confirmada depois, e isso muda o treino e a avaliação. Diretamente relevante para o MED, que confirma fraude dias depois.

Dal Pozzolo et al. (2014), *Learned lessons in credit card fraud detection from a practitioner perspective*, Expert Systems with Applications, 41(10). Sobre métricas de avaliação.

- **Carcillo et al. (2017), *Scarff*, framework de streaming com Spark.** Referência para processamento contínuo.
- **Lucas et al. (2019), *Multiple perspectives HMM-based feature engineering*, SAC 2019.** Modelos ocultos de Markov para representar sequências de transações como features.
- **Pizzolato et al. (2025), A Taxonomy of Pix Fraud in Brazil, arXiv 2511.20902.** Revisão da literatura combinada com entrevistas. Lista 15 metodologias de ataque e aponta autenticação multifator, monitoramento comportamental e detecção por IA como mitigações. Pede datasets e simuladores.
- **Zeimarani et al. (2026), PixSim, arXiv 2609.30684**. Simulador calibrado com dados abertos do BCB. Modela liquidação irreversível e janela de recuperação do MED, com política de três ações sob capacidade limitada de analistas. Útil para avaliar o seu motor sem dados rotulados.

Métodos

Desbalanceamento Extremo de Classes: A grande maioria das transações é legítima (frequentemente > 99,9%). Técnicas padrão de aprendizado de máquina podem ter acurácia de 99,9% simplesmente prevendo "tudo como legítimo", o que é inútil.
O efeito colateral: O *undersampling* altera artificialmente o *prior* (taxa basal) da base. O modelo passa a cuspir scores inflados (uma probabilidade predita de 0,60 pode corresponder, no mundo real, a um risco real de apenas 0,005).
Aplicação prática: Uso obrigatório da fórmula de ajuste de Bayes apresentada no artigo:

Custo Assimétrico dos Erros (*Cost-sensitive learning*): O custo de um falso positivo (bloquear o cartão de um cliente bom, gerando insatisfação) é muito diferente do custo de um falso negativo (permitir a fraude e arcar com o prejuízo financeiro). O modelo deve otimizar custos operacionais e financeiros, não métricas estatísticas puras como acurácia ou erro quadrático.

Computação assíncrona / Quase tempo real ou lote (*Near-line* / *Cold Path*): Acontece em segundo plano (milissegundos após o envio, a cada minuto ou a cada poucas horas), atualizando o perfil de risco do cliente, da chave Pix e do recebedor.

Modelos Clássicos de ML (LightGBM, XGBoost, CatBoost): Viabilidade no Pix: Totalmente viáveis e constituem a espinha dorsal da decisão síncrona.

Redes Neurais Recorrentes (RNNs / LSTMs / GRUs / Transformers)

- **Viabilidade no Pix: Viáveis com ressalvas de arquitetura.**
- **O desafio:** Executar uma LSTM ou GRU profunda em tempo real para cada transação pode introduzir uma latência de 20 a 80 ms, o que consome quase toda a margem de tempo do gateway transacional. Além disso, a infraestrutura exige GPUs caras para sustentar o throughput de picos de Pix.
- **Geração de Embeddings Comportamentais (Assíncrono):** A rede neural temporal não precisa dar a resposta final da transação. Em vez disso, a RNN roda em segundo plano conforme as ações ocorrem, gerando um vetor numérico (*embedding*) que resume o padrão temporal recente daquele usuário. Esse vetor é salvo no Redis/Feature Store e o modelo clássico (LightGBM) apenas lê esse embedding em milissegundos para tomar a decisão final.

Redes Neurais em Grafos (GNNs)

- **Viabilidade no Pix: Inviáveis para inferência síncrona completa ponta a ponta, mas ESSENCIAIS na análise estrutural e quase tempo real.**

Como as fintechs e bancos usam GNNs no Pix:

1. **Detecção de "Contas Laranja" (*Mule Accounts*):** A GNN é executada a cada poucos minutos ou horas para varrer a topologia de transferências. Ela detecta padrões clássicos de lavagem/fraude de Pix, como:
    - *Padrão Funil:* Múltiplas contas pequenas transferindo Pix quase simultâneos para uma única conta centralizadora.
    - *Padrão Dispersão:* Uma conta que recebe um Pix de alto valor e o fragmenta imediatamente em 10 transferências menores para chaves diferentes.
2. **Pontuação Pré-calculada de Risco da Chave / Conta Recebedora:** O modelo de grafo atribui continuamente uma "reputação de rede" para cada conta e chave Pix do sistema. Quando o usuário digita a chave para

transferir, o motor de regras síncrono consulta no cache o *score de risco de grafo* do recebedor (que já foi previamente inferido pela GNN).

3. **Grafos Subamostrados (Inductive / GraphSAGE):** Algumas infraestruturas modernas usam *GraphSAGE* limitando rigidamente a busca a 1 grau e no máximo 10 nós vizinhos mais recentes para manter a latência abaixo de 30 ms em GPUs dedicadas.

Técnica	**Papel no Pipeline do Pix**	**Local de Execução**	**Latência Típica**
Regras Rígidas + Listas	Filtros de segurança básicos (IP bloqueado, chave em lista negra do DICT, limite noturno).	Síncrono (Primeira linha)	0.5 – 2 ms
Ensembles (LightGBM / XGBoost)	**Decisão final de autorização / bloqueio / retenção cautelar.**	Síncrono (Hot Path)	2 – 8 ms
RNNs / LSTMs / Transformers	Detecção de quebra de sequência de eventos no app (comportamento atípico na sessão).	Quase síncrono ou gerador de *embeddings* temporais	15 – 50 ms
GNNs (Graph Neural Networks)	**Identificação de quadrilhas, anéis de fraude e contas laranja.**	Assíncrono / Quase tempo real (Micro-batches ou pós-transação)	100 ms – vários segundos
Isolation Forest	Monitoramento não supervisionado de desvios em novos	Pré-filtro ou auditoria de alertas	1 – 10 ms

	comportamentos ou anomalias de sistema.		

Capacidade da mesa de análise manual / Alertas: O banco tem um limite fixo de analistas para avaliar chamadas suspeitas ou revisar retenções cautelares (que o BACEN permite segurar por até 30 ou 60 minutos). O modelo não deve focar apenas em corte de probabilidade estático, mas em $P@k$

Elliptic Data Set (Kaggle/MIT)

- **O que é: O *dataset* de grafos financeiros mais famoso do mundo. É focado em transações de Bitcoin, mapeando mais de 200 mil transações (nós) e suas dependências (arestas).**
- **Por que usar no projeto Pix: O Bitcoin e o Pix compartilham a natureza de "conta para conta" instantânea. Este dataset possui uma coluna de rótulo onde as transações são marcadas como "ilícitas" (lavagem de dinheiro, ransomware) ou "lícitas", sendo perfeito para validar se sua implementação do GraphSAGE consegue separar a topologia de fraudadores da de usuários normais.**

PaySim (Kaggle)

- **O que é: Um simulador de dinheiro móvel (*mobile money*) criado com base em registros de um mês de um operador financeiro real na África.**
- **Por que usar no projeto Pix: Ele gera dados em formato tabular clássico (`origem`, `destino`, `valor`, `tipo`), totalizando mais de 6 milhões de linhas. Como a estrutura origem/destino está lá, você pode facilmente escrever um script em Python usando bibliotecas como `NetworkX` ou `DGL` (Deep Graph Library) para converter essas tabelas em um grafo bipartite e aplicar o GraphSAGE.**

Engenharia de Features Sequenciais via HMM (Lucas et al., 2019)

- **O problema no Pix: Redes neurais recorrentes (LSTMs) têm latência alta para rodar síncronas no Pix. Árvores tabulares (LightGBM) são rápidas, mas cegas à ordem exata dos eventos.**
- **Aplicação prática:**
    - **Modelos Ocultos de Markov (HMMs) modelam a probabilidade de transição entre estados comportamentais (ex.: "login habitual" →**

→ "troca de limite" →→ "cadastramento de chave" →→ "Pix de valor máximo")

A probabilidade dessa sequência sob o HMM do usuário vira uma feature numérica tabular única (ex.: `seq_anomaly_likelihood`).
O LightGBM consome essa feature pré-computada em menos de 1 ms, capturando a dinâmica temporal da sessão sem pagar a latência de inferir uma rede neural no *hot path*.

OPCIONALLL: Arquiteturas de Streaming em Tempo Real: SCARFF (Carcillo et al., 2017)

- **O problema no Pix: Como calcular contadores temporais ("quantos Pix esse recebedor recebeu nos últimos 5 minutos?", "qual o volume total transacionado nessa chave na última meia hora?") em escala de dezenas de milhares de TPS?**
- **Aplicação prática:**
    - **O framework SCARFF (baseado em Spark Streaming / Kafka) é o padrão de arquitetura de Feature Store em Streaming (hoje frequentemente implementado com Kafka + Flink + Redis).**
    - **Enquanto o gateway do Pix roda a inferência com LightGBM, a esteira de streaming atualiza assincronamente as janelas deslizantes que alimentam a *Feature Store*.**

Avaliação Semântica (JEV / System-1 AI)

Apenas o grupo restrito de transações que formam a "zona cinzenta" — ou seja, passaram pelas regras básicas, o grafo não encontrou triangulação óbvia, mas o valor e o contexto são atípicos (exemplo clássico: um sequestro relâmpago ou golpe do falso gerente) — chega ao JEV. Aqui, temos uma janela de 150 a 300 milissegundos.

- **O Problema dos LLMs Tradicionais: Modelos como GPT-4 geram texto "token a token" (autorregressão). Se você pedir para ele avaliar uma fraude, ele "escreve" a resposta internamente antes de dar o veredito. Esse processo é pesado, custoso e frequentemente excede 1 segundo — inaceitável para o SPI do BACEN. Além disso, LLMs podem sofrer de *hallucination* no esquema, quebrando o JSON de resposta que a API do banco espera.**
- **Contextualização de Alta Complexidade: O JEV consegue ler metadados sutis — por exemplo, que o celular está com a bateria em 5%, a transação está sendo feita às 3 da manhã em uma rodovia fora da cidade do usuário, para um recebedor recém-criado, mas de um valor**

quebrado (R$ 4.982,00). O JEV entende essa semântica de engenharia social ou coerção rapidamente, emitindo uma ordem de Bloqueio Cautelar para investigação humana na manhã seguinte.

- **Inferência de Passagem Única (Single Forward Pass) e decisão determinística: O JEV avalia o estado completo da transação em uma única passagem pela rede neural e retorna diretamente uma matriz de probabilidades associadas a opções pré-definidas (um *schema* fixo).**

A implementação exige a construção de um JSON de estado abrangente:

- **Dados Transacionais Clássicos: Valor, horário, conta de origem, conta de destino.**
- **Metadados do Dispositivo (Device Signals): Porcentagem de bateria atual, brilho da tela, uso recente de aplicativos, se o celular está sendo carregado, localização GPS.**
- **Contexto Comportamental (Opcional): O usuário abriu o app do banco após receber uma mensagem SMS contendo um link? A transação foi iniciada poucos minutos após o desbloqueio do aparelho em uma localização atípica?**
- **Montagem de Pares (Input/Output): A equipe de Risco cria dezenas de milhares de exemplos curados simulando o que a API vai receber e o que ela deve responder.**
- **Inclusão de "Hard Negatives": É vital incluir exemplos que parecem fraude, mas não são (falsos positivos).**

IEEE-CIS Fraud Detection Dataset

- **O que é: Criado pela Vesta Corporation, este dataset contém dados reais e ofuscados de transações de e-commerce e pagamentos online.**
- **Por que usar no projeto Pix: Ele é extremamente rico em metadados (*Device Type*, *Device Info*, distância, domínios de email). Você pode usar esses dados para gerar os "pares de treinamento" (Input/Output). O script pegaria essas colunas e as formataria em um JSON simulando o contexto situacional de um Pix, ensinando o modelo a associar tipos específicos de aparelhos ou IPs distantes ao *score* de fraude.**

Credit Card Fraud Detection (Kaggle - Grupo ULB): Contém 284 mil transações, mas os atributos numéricos (V1 a V28) foram transformados via PCA (Análise de Componentes Principais) por confidencialidade. Serve para treinar *Machine Learning* clássico (Random Forest, XGBoost), mas não para modelos semânticos (que precisam do contexto textual legível, como "São Paulo" ou "iPhone 13").

Inutilidade de olhar apenas para o pagador: Se a vítima foi coagida ou enganada, os dados do dispositivo são legítimos. O modelo precisa avaliar a conta de destino (chave Pix recebedora, tempo de abertura da conta de destino, padrão de recebimento).

GERADOR DE TESTES: PixSim: Simulator Calibrated with BCB Open Data

Jev ou System-One qualquer para analise semantica do texto do PIX????

Plano de implementação para 24 horas

O enunciado fornece um payload com dados de conta, de aparelho e de geolocalização, mas nenhum rótulo histórico. Isso define a estratégia: o núcleo é estatística e detecção de anomalias sobre o histórico que a própria API acumula, com regras explícitas. Supervisionado entra só com dados simulados. O prazo final é domingo às 12:00, então a meta é um sistema que roda de ponta a ponta com um modelo simples bem justificado.

1. Features derivadas do enunciado

Cada feature usa campos do payload e o histórico do banco próprio. A coluna de validação indica onde o método aparece na literatura. Quando a fonte é cinzenta, está marcada.

Feature	Cálculo	Campos do enunciado	Validação na literatura
Idade da conta de destino	`transaction_date - destination_account.opening_date` (dias)	`opening_date`	Art. 39-B §1º, II, do Regulamento do Pix, que lista o tempo desde a abertura da conta entre os fatores da avaliação de suspeita. Cinzento: mula em conta recém-aberta (Oscilar, FluxForce)

Idade da conta de origem	Mesmo cálculo para `source_account`	`source_account.opening_date`	Mesma base regulatória. Conta antiga com comportamento novo é sinal de conta tomada (Pizzolato et al., arXiv 2511.20902)
Desvio de valor do pagador	z robusto: `(log(amount) - mediana) / MAD` sobre o histórico do pagador	`amount`, histórico	Perfil do usuário em Bolton & Hand (2002). Uso de mediana e MAD porque o histórico é pequeno e assimétrico
Velocidade	Nº de transações do pagador nas últimas 1h e 24h	`transaction_date`, `source_account`	Regras de velocidade citadas em arXiv 2506.10842. Regra de velocity é o primeiro exemplo do enunciado
Montante acumulado	Soma do pagador em 24h	`amount`	Exemplo “Somou demais?” do enunciado; taxonomia de limites em Bolton & Hand

Recorrência do par	Fração das transferências do pagador que já foram para este destino	`source_account`, `destination_account`	Art. 39-B §1º, IV, que cita "recorrência" no perfil do pagador. Exemplo "Contraparte" do enunciado
Fan-in do destino	Nº de pagadores distintos que enviaram ao destino em 24h	`destination_account`	Subgrafos de fan-in em arXiv 2103.03227 e 2607.17586 (grafo local, sem GNN)
Dispositivo novo	`device_id` nunca visto nesta conta	`source.device_id`	Sinal de conta tomada: Fingerprint e BioCatch (cinzento). Taxonomia Pizzolato lista comprometimento de credencial
Dispositivo compartilhado	Nº de contas distintas usando o mesmo `device_id`	`device_id`	Mula operada de um aparelho (Fingerprint, cinzento)
Distância geográfica	Haversine entre `geolocation` atual e mediana histórica da conta	`geolocation` (opcional)	Viagem impossível em Adaptive Security e Bureau ID (cinzento)

Horário atípico	**Frequência do horário na conta, com suavização**	**`transaction_date`**	**BioCatch (cinzento) indica que golpes concentram-se em horário comercial, então não usar horário fixo**
Benford por conta	**Teste χ^2 do primeiro dígito das somas diárias**	**`amount`, agregados**	**Milliman (VGZ) e Branets (Univ. Tartu). Evidência fraca, uso complementar**

2. Modelos, em ordem de implementação

M0. Regras determinísticas (implementar primeiro, 2 horas).
Listas de bloqueio em memória (chave e conta já marcadas), limites fixos por faixa de conta nova, teto de velocidade e teto de montante. Cada regra tem ID, parâmetro e motivo, que vão para o `metadata`.

- **Validação: pré-filtro sem aprendizado antes do modelo, descrito em Springer (2023, s40854-023-00470-w). Reporta menos perda que regras estáticas sozinhas.**

M1. Escores estatísticos por conta (4 horas).

- **Velocidade com Poisson–Gamma: taxa de transações por conta com prior Gamma(α, β). O escore é a probabilidade de observar a contagem da janela sob a taxa esperada. Dá um escore para conta nova, sem limiar arbitrário.**
- **Desvio de valor com z robusto.**
- **Verificação de dispersão: antes de usar Poisson, comparar variância e média das contagens. Se a variância for muito maior que a média, trocar por binomial negativa.**
- **Validação: perfil estatístico em Bolton & Hand (2002). Verificação de dispersão é teste padrão de contagens; o enunciado não exige o teste, mas ele evita escolha errada de distribuição.**

M2. Isolation Forest sobre o vetor de features (3 horas).
`sklearn.ensemble.IsolationForest` com `contamination` calibrado pelo custo, não por padrão.

- **Validação: comparação supervisionado vs não supervisionado em arXiv 1904.10604. Isolation Forest é a referência de comparação em arXiv 2506.10842 e no caso Milliman.**
- **Treino: como não há histórico rotulado, treinar com transações simuladas. Se o repositório do PixSim estiver disponível, usar o simulador (arXiv 2609.30684) para gerar dados com fraude injetada. Caso contrário, gerar dados sintéticos com distribuições dos campos do enunciado.**

M3. Supervisionado calibrado (só se houver rótulos simulados, 3 horas).
Regressão logística como baseline e gradient boosting (LightGBM) como segundo modelo. Calibração por Platt ou isotônica depois do treino.

- **Validação: Dal Pozzolo et al. (2015, CIDM) mostra que undersampling desloca as probabilidades, então a calibração é obrigatória. Dal Pozzolo et al. (2018, TNNLS) trata rótulos atrasados, o que se aplica ao MED.**
- **Limitação grave: rótulos gerados pelas próprias regras criam circularidade. Isso precisa ser dito na apresentação.**

M4. Combinação e limiares (2 horas).
Score final = combinação das saídas de M0, M1 e M2, por média ponderada ou regressão logística sobre os escores. Limiares de dois cortes:

- **`two_factor_required` quando o score passa o limiar baixo;**
- **`rejected` quando passa o alto, ou quando uma regra dura dispara.**
- **Validação: escolha de limiar com custo assimétrico em arXiv 2601.07276 e alerta em arXiv 2608.16147, que mostra que muitos trabalhos não reportam o limiar. Reporte o limiar e a função de custo no README.**

3. Análises estatísticas a rodar

1. **Distribuição de `amount` por pagador: histograma e QQ-plot de log(amount) para justificar o z robusto.**
2. **Teste de dispersão das contagens: média vs variância por conta, para escolher Poisson ou binomial negativa.**
3. **Benford sobre somas diárias por conta: χ^2 e distância de Kolmogorov, como feature complementar.**
4. **Curva precisão-recall e custo esperado: sobre os dados simulados, com o custo de falso negativo maior que o de falso positivo. Validação: arXiv 2601.07276.**
5. **Calibração: curva de confiabilidade e Brier score se houver M3.**

6. **Estabilidade de features (PSI): comparar a distribuição das features entre duas janelas simuladas para detectar drift. Validação: concept drift em Dal Pozzolo et al. (2015, IJCNN).**
7. **Sensibilidade de limiar: variar os limiares e mostrar a taxa de transações em cada uma das três faixas. Isso é o que a banca vai perguntar sobre fricção.**

4. Implementação do endpoint

- **POST /pix/transaction: valida o payload, calcula as features com consultas SQL indexadas (`source_account`, `destination_account`, `device_id`, `transaction_date`), roda M0, M1 e M2, grava a transação e a análise, e retorna `key`, `status` e `metadata`.**
- **GET /pix/transaction/{id}: devolve a análise gravada.**
- **Banco: Postgres ou SQLite. Tabelas: `transactions`, `analyses`, `accounts` (com `opening_date`), `devices` (com `first_seen`). Índices nas colunas usadas pelas janelas.**
- **Latência: todas as consultas com índice, sem chamada externa. Medir o P95 do endpoint em teste local e reportar.**
- **Metadata: lista de regras disparadas, z-scores, percentis das features e a versão do modelo. Isso atende ao requisito de motivo explicável e dá base para o art. 20 da LGPD sobre revisão de decisões automatizadas.**
- **Testes com pytest: três cenários fixos: pagador normal, conta de destino recém-aberta com fan-in alto (mula), e dispositivo novo com geolocalização distante (conta tomada). Cada um deve produzir a classificação esperada.**